

Phase 6: Recursive Tree Analysis Algorithms

0. Purpose of This Phase

Phase 6 teaches how to use recursion to **analyze** a binary tree.

In earlier phases, you learned how to:

- Understand tree vocabulary.
- Store a binary tree using pointers.
- Build a binary tree.
- Traverse a tree using recursion.
- Traverse a tree using stacks and queues.

Now you will use the same traversal idea to answer useful questions about a tree.

In Phase 6, we are not only printing nodes. We are calculating information from the tree.

The main beginner idea is:

Traversal is not only for printing.
Traversal can also be used to calculate answers.

1. Current Progress Analysis

Before Phase 6, you already studied Phases 1 to 5.

Phase	What you learned	Why it matters in Phase 6
Phase 1	Tree basics: root, child, leaf, internal node, degree, height	Phase 6 asks questions about nodes, children, and height
Phase 2	Binary tree formulas and degree concepts	Phase 6 counts degree-2 nodes
Phase 3	Linked representation using <code>Node*</code> , <code>lchild</code> , <code>rchild</code> , and <code>nullptr</code>	Phase 6 functions follow these child pointers
Phase 4	Recursive preorder, inorder, and postorder traversal	Phase 6 uses the same recursive thinking
Phase 5	Iterative traversal, stacks, queues, and level order	You now understand that traversal can be controlled in different ways

So your current position is:

You know how to visit every node in a tree.
Now you will learn how to use those visits to calculate useful results.

Important bridge:

Before Phase 6:
You used traversal to print nodes.

In Phase 6:
You use traversal to calculate answers.

That is the big upgrade in this phase.

2. What Phase 6 Covers

Phase 6 covers four important recursive tree analysis algorithms:

1. Count all nodes in a binary tree.
2. Count nodes with degree 2.
3. Sum all node values.
4. Calculate the height of a binary tree.

These are common beginner and interview-style tree problems.

They all use the same basic recursive structure:

Solve the left subtree.
Solve the right subtree.
Combine both answers with the current node.

3. Example Tree Used in This Phase

We will use this binary tree throughout the note:

```
    10
   /
  20  30
```

40 / /
50 50 60

3.1 Node Facts

The nodes are:

10, 20, 30, 40, 50, 60

So the total number of nodes is:

6

3.2 Degree-2 Node Facts

A degree-2 node has exactly two children.

In this tree:

10 has two children: 20 and 30
30 has two children: 50 and 60

So the degree-2 nodes are:

10 and 30

Therefore:

Number of degree-2 nodes = 2

3.3 Sum Facts

The sum of all node values is:

10 + 20 + 30 + 40 + 50 + 60 = 210

So:

Sum = 210

3.4 Height Facts

The longest root-to-leaf paths are:

10 -> 20 -> 40

and:

10 -> 30 -> 50

or:

10 -> 30 -> 60

Each longest path has 3 nodes.

So:

Height in nodes = 3

Each longest path has 2 edges.

So:

Height in edges = 2

4. Important Node Structure

In Phase 6, we use a linked binary tree node.

```
struct Node {  
    int data;  
    Node* lchild;  
    Node* rchild;
```

```
Node(int val) {  
    data = val;  
    lchild = nullptr;  
    rchild = nullptr;  
}  
};
```

4.1 Meaning of Each Part

```
int data;
```

This stores the value inside the node.

Example:

```
10, 20, 30, etc.
```

```
Node* lchild;
```

This stores the address of the left child.

```
Node* rchild;
```

This stores the address of the right child.

```
lchild = nullptr;  
rchild = nullptr;
```

This means the node starts with no children.

4.2 Why `nullptr` Is Important

`nullptr` means:

```
There is no node here.
```

For example, if a node has no left child:

```
p->lchild == nullptr
```

If a node has no right child:

```
p->rchild == nullptr
```

Every recursive function in this phase must check for `nullptr`.

5. The Main Recursive Pattern

Almost every Phase 6 function follows this pattern:

```
if (p == nullptr) {  
    return 0;  
}  
  
int x = function(p->lchild);  
int y = function(p->rchild);  
  
return something using x, y, and p;
```

5.1 Simple Meaning

This means:

```
If there is no node, return 0.  
Otherwise:  
    Ask the left subtree for its answer.  
    Ask the right subtree for its answer.  
    Combine those answers with the current node.
```

5.2 Why We Return 0 for `nullptr`

If there is no node, it contributes nothing.

For counting nodes:

```
No node means count 0.
```

For summing values:

No node means sum 0.

For height in nodes:

No node means height 0.

So returning `0` is natural for these functions.

5.3 Why This Is Postorder-Style Recursion

In Phase 4, postorder traversal followed this order:

Left, Right, Root

Phase 6 functions often work like postorder because the current node waits for answers from both subtrees first.

The node says:

First, I need the left answer.
Then, I need the right answer.
Then, I can calculate my own final answer.

So Phase 6 is not always printing postorder, but the calculation style is similar to postorder.

Part A: Counting All Nodes

6. What Does Counting Nodes Mean?

Counting all nodes means finding how many real nodes exist in the tree.

For this tree:

```
    10
   /
  20  30
```

```
    /    /
   40   50  60
```

The nodes are:

```
10, 20, 30, 40, 50, 60
```

So:

```
Total nodes = 6
```

7. Beginner Idea for Counting Nodes

To count nodes from any current node:

```
Total nodes = nodes in left subtree + nodes in right subtree + 1
```

The `+1` means:

```
Count the current node itself.
```

Memory trick:

```
Left count + Right count + Myself
```

8. Code for Counting Nodes

```
int countNodes(Node* p) {
    if (p == nullptr) {
        return 0;
    }

    int x = countNodes(p->lchild);
    int y = countNodes(p->rchild);
```



```
    return x + y + 1;
}
```

9. Explanation of the Code

9.1 Base Case

```
if (p == nullptr) {
    return 0;
}
```

If there is no node, the count is 0.

This stops the recursion safely.

9.2 Left Subtree Count

```
int x = countNodes(p->lchild);
```

This asks:

How many nodes are in my left subtree?

9.3 Right Subtree Count

```
int y = countNodes(p->rchild);
```

This asks:

How many nodes are in my right subtree?

9.4 Final Return

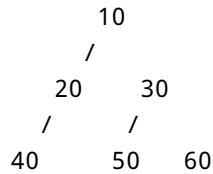
```
return x + y + 1;
```

This means:

```
left count + right count + current node
```

10. Dry Run: Counting Nodes

Use this tree:



10.1 Leaf Node 40

Node 40 has no children.

```
countNodes(40)

left side = nullptr -> returns 0
right side = nullptr -> returns 0

return 0 + 0 + 1 = 1
```

So:

```
countNodes(40) = 1
```

A leaf returns 1 because it counts itself.

10.2 Node 20

Node 20 has one left child, 40, and no right child.

```
countNodes(20)

left answer = countNodes(40) = 1
right answer = nullptr = 0

return 1 + 0 + 1 = 2
```

So:

```
countNodes(20) = 2
```

The subtree rooted at 20 has 2 nodes:

20 and 40

10.3 Node 30

Node 30 has two children: 50 and 60.

```
countNodes(30)

left answer = countNodes(50) = 1
right answer = countNodes(60) = 1

return 1 + 1 + 1 = 3
```

So:

```
countNodes(30) = 3
```

The subtree rooted at 30 has 3 nodes:

30, 50, 60

10.4 Root Node 10

Root 10 has left subtree 20 and right subtree 30.

```
countNodes(10)

left answer = countNodes(20) = 2
right answer = countNodes(30) = 3

return 2 + 3 + 1 = 6
```

Final answer:

Total nodes = 6

Part B: Counting Degree-2 Nodes

11. What Is a Degree-2 Node?

A degree-2 node is a node that has exactly two children.

That means:

left child exists AND right child exists

11.1 Example of a Degree-2 Node

```
    10
   /
  20  30
```

Node 10 has two children:

left child = 20
right child = 30

So node 10 is a degree-2 node.

11.2 Example That Is Not Degree-2

```
    20
   /
  40
```

Node 20 has only one child.

So node 20 is not degree-2.

11.3 Leaf Node

40

Node 40 has no children.

So node 40 is not degree-2.

12. Beginner Idea for Counting Degree-2 Nodes

The structure is similar to counting all nodes.

But this time, we only add 1 when the current node has both children.

```
Degree-2 count = degree-2 nodes in left subtree
                  + degree-2 nodes in right subtree
                  + 1 only if current node has two children
```

Memory trick:

```
Left answer + Right answer + maybe myself
```

The word **maybe** is important.

We count the current node only if it has both children.

13. Code for Counting Degree-2 Nodes

```
int countDegreeTwo(Node* p) {
    if (p == nullptr) {
        return 0;
    }

    int x = countDegreeTwo(p->lchild);
    int y = countDegreeTwo(p->rchild);

    if (p->lchild != nullptr && p->rchild != nullptr) {
        return x + y + 1;
    }
}
```

```
    return x + y;  
}
```

14. Explanation of the Code

14.1 Base Case

```
if (p == nullptr) {  
    return 0;  
}
```

If there is no node, there is no degree-2 node.

So return 0.

14.2 Left and Right Answers

```
int x = countDegreeTwo(p->lchild);  
int y = countDegreeTwo(p->rchild);
```

This asks both subtrees:

How many degree-2 nodes do you contain?

14.3 Checking the Current Node

```
if (p->lchild != nullptr && p->rchild != nullptr)
```

This checks if both children exist.

The operator `&&` means:

AND

So the condition means:

left child exists AND right child exists

14.4 If Current Node Is Degree-2

```
return x + y + 1;
```

This means:

```
left degree-2 count + right degree-2 count + current node
```

14.5 If Current Node Is Not Degree-2

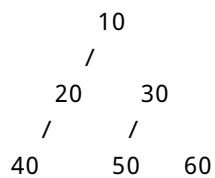
```
return x + y;
```

This means:

Only return the left and right answers.
Do not count the current node.

15. Dry Run: Counting Degree-2 Nodes

Use this tree:



Check each node:

Node	Left child exists?	Right child exists?	Degree-2?
10	Yes	Yes	Yes
20	Yes	No	No
30	Yes	Yes	Yes
40	No	No	No
50	No	No	No

Node	Left child exists?	Right child exists?	Degree-2?
60	No	No	No

So the degree-2 nodes are:

10 and 30

Final answer:

Degree-2 nodes = 2

16. Important Warning About Degree-1 Nodes

A degree-1 node has exactly one child.

Example:

```

      20
     /
    40

```

Node 20 has one child.

So:

20 is degree-1, not degree-2.

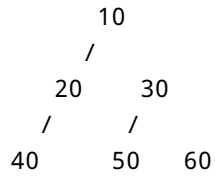
Do not count degree-1 nodes when the question asks for degree-2 nodes.

Part C: Summing All Node Values

17. What Does Summing Nodes Mean?

Summing all node values means adding the data stored in every node.

Use this tree:



The sum is:

$$10 + 20 + 30 + 40 + 50 + 60 = 210$$

So:

$$\text{Sum} = 210$$

18. Beginner Idea for Summing Nodes

Counting nodes adds `1` for each node.

Summing nodes adds the actual node value.

Counting nodes:

```
return x + y + 1;
```

Summing nodes:

```
return x + y + p->data;
```

Memory trick:

```
Left sum + Right sum + My data
```

19. Code for Summing Nodes

```
int sumOfNodes(Node* p) {  
    if (p == nullptr) {  
        return 0;  
    }  
  
    int x = sumOfNodes(p->lchild);  
    int y = sumOfNodes(p->rchild);  
  
    return x + y + p->data;  
}
```

20. Explanation of the Code

20.1 Base Case

```
if (p == nullptr) {  
    return 0;  
}
```

If there is no node, it contributes 0 to the sum.

20.2 Left Sum

```
int x = sumOfNodes(p->lchild);
```

This asks:

What is the sum of my left subtree?

20.3 Right Sum

```
int y = sumOfNodes(p->rchild);
```

This asks:

What is the sum of my right subtree?

20.4 Final Return

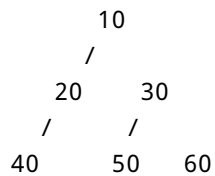
```
return x + y + p->data;
```

This means:

```
left sum + right sum + current node value
```

21. Dry Run: Summing Nodes

Use this tree:



21.1 Leaf Node 40

```
sumOfNodes(40)

left sum = 0
right sum = 0
current data = 40

return 0 + 0 + 40 = 40
```

So:

```
sumOfNodes(40) = 40
```

21.2 Node 20

```
sumOfNodes(20)
```

```
left sum = 40
right sum = 0
current data = 20

return 40 + 0 + 20 = 60
```

So:

```
sumOfNodes(20) = 60
```

21.3 Node 30

```
sumOfNodes(30)

left sum = 50
right sum = 60
current data = 30

return 50 + 60 + 30 = 140
```

So:

```
sumOfNodes(30) = 140
```

21.4 Root Node 10

```
sumOfNodes(10)

left sum = 60
right sum = 140
current data = 10

return 60 + 140 + 10 = 210
```

Final answer:

```
Sum = 210
```

Part D: Calculating Height Recursively

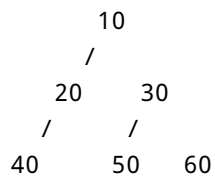
22. What Is Height?

Height means the length of the longest downward path from a node to a leaf.

In simple words:

Height tells how tall the tree is.

Use this tree:



The longest paths are:

10 -> 20 -> 40
10 -> 30 -> 50
10 -> 30 -> 60

Each longest path has:

3 nodes
2 edges

So:

Height in nodes = 3
Height in edges = 2

23. Two Height Conventions

There are two common ways to measure height.

Convention	Empty tree	Single node	Meaning
Height in nodes	0	1	Counts nodes on the longest path
Height in edges	-1 or derived	0	Counts edges on the longest path

23.1 Height in Nodes

Height in nodes counts how many nodes are on the longest path.

Example:

```
10 -> 20 -> 40
```

This path has 3 nodes.

So:

```
Height in nodes = 3
```

23.2 Height in Edges

Height in edges counts how many edges are on the longest path.

Example:

```
10 -> 20 -> 40
```

This path has 2 edges:

```
10 -> 20
20 -> 40
```

So:

```
Height in edges = 2
```

23.3 Relationship Between Them

For a non-empty tree:

```
height in edges = height in nodes - 1
```

Example:

```
height in nodes = 3  
height in edges = 3 - 1 = 2
```

24. Beginner Idea for Calculating Height

For height, we do not add the left and right answers.

Instead, we choose the bigger answer.

Why?

Because height is about the longest path.

```
Height = bigger of left height and right height + 1
```

The `+1` means:

```
Include the current node.
```

Memory trick:

```
Height follows the taller side.
```

25. Code for Height in Nodes

```
int heightInNodes(Node* p) {  
    if (p == nullptr) {  
        return 0;  
    }  
  
    int x = heightInNodes(p->lchild);  
    int y = heightInNodes(p->rchild);
```

```
    return (x > y ? x : y) + 1;
}
```

26. Explanation of the Code

26.1 Base Case

```
if (p == nullptr) {
    return 0;
}
```

If there is no node, the height in nodes is 0.

26.2 Left Height

```
int x = heightInNodes(p->lchild);
```

This asks:

What is the height of my left subtree?

26.3 Right Height

```
int y = heightInNodes(p->rchild);
```

This asks:

What is the height of my right subtree?

26.4 Choosing the Bigger Side

```
return (x > y ? x : y) + 1;
```

This means:

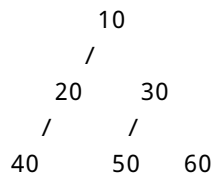
If x is bigger, use x.
Otherwise, use y.
Then add 1 for the current node.

This is the same idea as:

```
return max(x, y) + 1;
```

27. Dry Run: Height in Nodes

Use this tree:



27.1 Leaf Node 40

```
heightInNodes(40)

left height = 0
right height = 0

return max(0, 0) + 1 = 1
```

So:

```
heightInNodes(40) = 1
```

A leaf has height 1 in node-count style.

27.2 Node 20

```
heightInNodes(20)

left height = 1
right height = 0
```

```
return max(1, 0) + 1 = 2
```

So:

```
heightInNodes(20) = 2
```

27.3 Node 30

```
heightInNodes(30)  
  
left height = 1  
right height = 1  
  
return max(1, 1) + 1 = 2
```

So:

```
heightInNodes(30) = 2
```

27.4 Root Node 10

```
heightInNodes(10)  
  
left height = 2  
right height = 2  
  
return max(2, 2) + 1 = 3
```

Final answer:

```
Height in nodes = 3
```

Height in edges:

```
Height in edges = height in nodes - 1  
Height in edges = 3 - 1 = 2
```

So:

Height in edges = 2

Part E: Complete Phase 6 Program

28. Complete C++ Program

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* lchild;
    Node* rchild;

    Node(int val) {
        data = val;
        lchild = nullptr;
        rchild = nullptr;
    }
};

int countNodes(Node* p) {
    if (p == nullptr) {
        return 0;
    }

    int x = countNodes(p->lchild);
    int y = countNodes(p->rchild);

    return x + y + 1;
}

int countDegreeTwo(Node* p) {
    if (p == nullptr) {
        return 0;
    }

    int x = countDegreeTwo(p->lchild);
    int y = countDegreeTwo(p->rchild);

    if (p->lchild != nullptr && p->rchild != nullptr) {
```

```

        return x + y + 1;
    }

    return x + y;
}

int sumOfNodes(Node* p) {
    if (p == nullptr) {
        return 0;
    }

    int x = sumOfNodes(p->lchild);
    int y = sumOfNodes(p->rchild);

    return x + y + p->data;
}

int heightInNodes(Node* p) {
    if (p == nullptr) {
        return 0;
    }

    int x = heightInNodes(p->lchild);
    int y = heightInNodes(p->rchild);

    return (x > y ? x : y) + 1;
}

int main() {
    Node* root = new Node(10);

    root->lchild = new Node(20);
    root->rchild = new Node(30);

    root->lchild->lchild = new Node(40);

    root->rchild->lchild = new Node(50);
    root->rchild->rchild = new Node(60);

    cout << "Total nodes: " << countNodes(root) << endl;
    cout << "Degree-2 nodes: " << countDegreeTwo(root) << endl;
    cout << "Sum of values: " << sumOfNodes(root) << endl;
    cout << "Height in nodes: " << heightInNodes(root) << endl;
    cout << "Height in edges: " << heightInNodes(root) - 1 << endl;

    return 0;
}

```

29. Tree Created by the Program

This part creates the root:

```
Node* root = new Node(10);
```

Tree now:

```
10
```

This part creates children of 10 :

```
root->lchild = new Node(20);  
root->rchild = new Node(30);
```

Tree now:

```
  10  
 /  
20  30
```

This part creates child of 20 :

```
root->lchild->lchild = new Node(40);
```

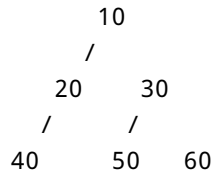
Tree now:

```
  10  
 /  
20  30  
 /  
40
```

This part creates children of 30 :

```
root->rchild->lchild = new Node(50);  
root->rchild->rchild = new Node(60);
```

Final tree:



30. Expected Output

Total nodes: 6
Degree-2 nodes: 2
Sum of values: 210
Height in nodes: 3
Height in edges: 2

Part F: Comparing the Four Functions

31. Function Comparison Table

Function	Base case	Left and right are combined by	Current node contributes
<code>countNodes</code>	<code>nullptr</code> returns 0	<code>x + y</code>	<code>+ 1</code>
<code>countDegreeTwo</code>	<code>nullptr</code> returns 0	<code>x + y</code>	<code>+ 1</code> only if both children exist
<code>sumOfNodes</code>	<code>nullptr</code> returns 0	<code>x + y</code>	<code>+ p->data</code>
<code>heightInNodes</code>	<code>nullptr</code> returns 0	bigger of <code>x</code> and <code>y</code>	<code>+ 1</code>

32. Simple Memory Table

Task	Formula idea
Count all nodes	<code>left + right + 1</code>
Count degree-2 nodes	<code>left + right + maybe 1</code>
Sum all values	<code>left + right + data</code>
Calculate height	<code>bigger(left, right) + 1</code>

Memory trick:

```
Count nodes    -> left + right + 1
Count degree-2 -> left + right + maybe 1
Sum nodes      -> left + right + data
Height         -> bigger(left, right) + 1
```

Part G: Complexity Analysis

33. Time Complexity

Each Phase 6 function visits every real node once.

So for a tree with `N` nodes:

```
Time complexity = O(N)
```

This applies to:

- `countNodes`
- `countDegreeTwo`
- `sumOfNodes`
- `heightInNodes`

Why?

Because the function must look at every node to calculate the final answer.

If there are 6 nodes, it visits 6 real nodes.

If there are 100 nodes, it visits 100 real nodes.

So the time grows with the number of nodes.

34. Space Complexity

Recursive functions use the system stack.

The maximum stack space depends on the height of the tree.

So:

Space complexity = $O(H)$

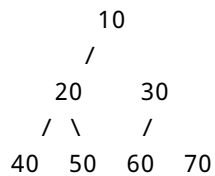
Here:

H = height of the tree

34.1 Balanced Tree Case

In a balanced tree, the height is smaller.

Example:

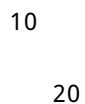


The recursion stack does not grow too deeply.

34.2 Skewed Tree Case

In a skewed tree, the height can be close to the number of nodes.

Example:



30

40

This behaves like a linked list.

In the worst case:

$H = N$

So the worst-case recursive stack space becomes:

$O(N)$

35. Complexity Summary

Function	Time complexity	Extra space
countNodes	$O(N)$	$O(H)$
countDegreeTwo	$O(N)$	$O(H)$
sumOfNodes	$O(N)$	$O(H)$
heightInNodes	$O(N)$	$O(H)$

Worst-case extra space for a skewed tree:

$O(N)$

Part H: Common Beginner Mistakes

36. Mistake 1: Forgetting the Null Base Case

Wrong:

```
int countNodes(Node* p) {  
    int x = countNodes(p->lchild);  
    int y = countNodes(p->rchild);  
    return x + y + 1;  
}
```

Problem:

If `p` is `nullptr`, this line crashes:

```
p->lchild
```

Correct:

```
if (p == nullptr) {  
    return 0;  
}
```

Always write the base case first.

37. Mistake 2: Adding Both Sides for Height

Wrong:

```
return x + y + 1;
```

This counts nodes.

It does not calculate height.

Correct:

```
return max(x, y) + 1;
```

or:

```
return (x > y ? x : y) + 1;
```

Why?

Because height follows only the longest path.

38. Mistake 3: Counting Degree-1 Nodes as Degree-2

Wrong:

```
if (p->lchild != nullptr || p->rchild != nullptr)
```

The operator `||` means OR.

This condition means:

```
left child exists OR right child exists
```

That would count nodes with one child too.

Correct:

```
if (p->lchild != nullptr && p->rchild != nullptr)
```

The operator `&&` means AND.

This condition means:

```
left child exists AND right child exists
```

That is exactly degree-2.

39. Mistake 4: Confusing Height in Nodes and Height in Edges

Consider this tree:

```
10
 /
20
```

Height in nodes:

```
10 -> 20 = 2 nodes
```

Height in edges:

```
10 -> 20 = 1 edge
```

So:

```
height in edges = height in nodes - 1
```

If `heightInNodes(root)` returns 2, then:

```
heightInEdges = heightInNodes(root) - 1;
```

So:

```
height in edges = 1
```

40. Mistake 5: Thinking All Recursive Functions Must Print Something

In Phase 4, recursive traversal printed nodes.

In Phase 6, recursive functions usually return answers.

Example:

```
return x + y + 1;
```

This function is not printing.

It is calculating and returning a value.

Important idea:

Traversal can print, but traversal can also return information.

41. Mistake 6: Not Returning a Value on Every Path

In C++, if a function has return type `int`, it must return an integer.

Wrong:

```
int countDegreeTwo(Node* p) {
    if (p == nullptr) {
        return 0;
    }

    int x = countDegreeTwo(p->lchild);
    int y = countDegreeTwo(p->rchild);

    if (p->lchild != nullptr && p->rchild != nullptr) {
        return x + y + 1;
    }
}
```

Problem:

If the node is not degree-2, there is no return statement.

Correct:

```
int countDegreeTwo(Node* p) {
    if (p == nullptr) {
        return 0;
    }

    int x = countDegreeTwo(p->lchild);
    int y = countDegreeTwo(p->rchild);

    if (p->lchild != nullptr && p->rchild != nullptr) {
        return x + y + 1;
    }

    return x + y;
}
```

Now every path returns a value.

Part I: Key Vocabulary

42. Phase 6 Vocabulary Table

Term	Simple meaning
Recursive tree analysis	Using recursion to calculate information about a tree
Base case	The stopping condition of recursion
<code>nullptr</code>	Means no node exists there
Count nodes	Find how many nodes are in the tree
Degree-2 node	A node with exactly two children
Sum of nodes	Add all node values together
Height in nodes	Number of nodes on the longest path
Height in edges	Number of edges on the longest path
Left subtree answer	The result returned from the left child
Right subtree answer	The result returned from the right child
Postorder-style calculation	Solve left and right first, then calculate current node
<code>O(N)</code>	Time grows with number of nodes
<code>O(H)</code>	Space grows with height of tree

Part J: Practice Questions

43. Practice Tree 1

Use this tree:

```
  5
 /
3  8
```

/ /
1 7 9

Answer these questions:

1. How many total nodes are there?
2. How many degree-2 nodes are there?
3. What is the sum of all values?
4. What is the height in nodes?
5. What is the height in edges?

43.1 Answers

Total nodes:

5, 3, 8, 1, 7, 9 = 6 nodes

So:

Total nodes = 6

Degree-2 nodes:

5 has two children: 3 and 8
8 has two children: 7 and 9

So:

Degree-2 nodes = 2

Sum:

5 + 3 + 8 + 1 + 7 + 9 = 33

So:

Sum = 33

Height in nodes:

Longest path: 5 -> 3 -> 1
or 5 -> 8 -> 7
or 5 -> 8 -> 9

Each longest path has 3 nodes.

So:

Height in nodes = 3

Height in edges:

Height in edges = 3 - 1 = 2

So:

Height in edges = 2

44. Practice Tree 2

Use this tree:

```
      1
     /
    2
   /
  3
 /
4
```

Answer these questions:

1. How many total nodes are there?
2. How many degree-2 nodes are there?
3. What is the sum of all values?

4. What is the height in nodes?
5. What is the height in edges?

44.1 Answers

Total nodes:

1, 2, 3, 4 = 4 nodes

So:

Total nodes = 4

Degree-2 nodes:

No node has two children.

So:

Degree-2 nodes = 0

Sum:

1 + 2 + 3 + 4 = 10

So:

Sum = 10

Height in nodes:

Longest path: 1 -> 2 -> 3 -> 4

This path has 4 nodes.

So:

Height in nodes = 4

Height in edges:

Height in edges = 4 - 1 = 3

So:

Height in edges = 3

45. Practice Tree 3

Use this tree:

```
      100
     /
    50   150
```

Answer these questions:

1. How many total nodes are there?
2. How many degree-2 nodes are there?
3. What is the sum of all values?
4. What is the height in nodes?
5. What is the height in edges?

45.1 Answers

Total nodes:

100, 50, 150 = 3 nodes

So:

Total nodes = 3

Degree-2 nodes:

100 has two children: 50 and 150

So:

Degree-2 nodes = 1

Sum:

$100 + 50 + 150 = 300$

So:

Sum = 300

Height in nodes:

Longest path: 100 -> 50
or 100 -> 150

Each longest path has 2 nodes.

So:

Height in nodes = 2

Height in edges:

Height in edges = $2 - 1 = 1$

So:

Height in edges = 1

Part K: Final Phase 6 Summary

46. Main Summary

By the end of Phase 6, you should understand that recursive tree traversal can be used to calculate answers.

You learned four important functions:

1. `countNodes`
2. `countDegreeTwo`
3. `sumOfNodes`
4. `heightInNodes`

They all follow the same basic idea:

Ask the left subtree.
Ask the right subtree.
Combine the answers with the current node.

47. Final Memory Tricks

47.1 Main Recursive Memory Trick

Left subtree, what is your answer?
Right subtree, what is your answer?
Now I combine both answers with myself.

47.2 Formula Memory Trick

Count nodes	-> <code>left + right + 1</code>
Count degree-2	-> <code>left + right + maybe 1</code>
Sum nodes	-> <code>left + right + data</code>
Height	-> <code>bigger(left, right) + 1</code>

47.3 Height Memory Trick

Height follows the taller side.

47.4 Degree-2 Memory Trick

Degree-2 means left child AND right child.

47.5 Null Memory Trick

nullptr means no node, so return 0.

48. What Comes Next

After Phase 6, you are ready for Phase 7.

Phase 7 studies **strict binary trees**.

A strict binary tree is a special binary tree where every node has either:

0 children

or:

2 children

No node is allowed to have exactly one child.

The recursion skills from Phase 6 will help you check whether a tree is strict.